The Round Table Exchange (RTE) — System Architecture & Test Coverage

A Multi-Party Skill Barter Network with Graph-Based Cycle Matching (v2.4)

Authors: Ahmmad Abdali Khan, Sumaia Bintey Ismail, Alin | Dept. of CSE, UITS | Date: 13 Sep 2026

1. System Architectural Overview

Core Algorithmic Engine: RTE solves multi-lateral trade imbalances by constructing a directed multigraph where nodes represent users and directed edges indicate skill offerings and desires. A Bounded-Depth DFS engine traverses exchange loops of length $k \in \{2, 3, 4\}$, avoiding exponential branch explosion. Cycles are dynamically scored using a composite multi-signal formula: $S = \alpha \cdot S_{geo} + (1 - \alpha) \cdot S_{sched}$, balancing Haversine geographic proximity with weekly timetable overlap.

Consensus & Dual-Engine Architecture: Implements an atomic 5-stage group state machine (*Proposed* → *Notified* → *Accepting* → *Locked* → *Executed*) with single-veto integrity. The system operates on a dual execution model: a high-performance **FastAPI** Python REST backend for enterprise servers, and a zero-dependency **In-Browser Client Engine** for serverless static hosting (GitHub Pages) with full algorithmic parity.

2. System Specifications & Technology Stack

Component	Primary Technologies	Architectural Responsibility
Frontend UI/UX	HTML5, CSS3 (Dark Cyber), Vanilla JS, Vis.js	Single-page dashboard, ForceAtlas2 physics canvas, real-time loop inspector, consensus UI
Backend Gateway	FastAPI, Uvicorn, Pydantic v2, Python 3.14	Asynchronous REST API, input schema validation, CORS security, in-memory state store
Graph & Matching	NetworkX, Bounded DFS, Haversine geodesics	Multigraph construction, cyclic path discovery, multi-signal candidate ranking
State Machine	Python Workflow Engine (in-memory & client)	Atomic multi-user proposal tracking, timeout expiration, group voting orchestration
Benchmarking Suite	Synthetic Dhaka Generator (20–500 nodes)	Empirical complexity verification: $O(n \cdot m^{k-1})$ latency & cycle count profiling

3. System Verification & Automated Test Coverage Report

Test Execution Status: 13 / 13 Automated Tests PASSED (100% Success Rate in 0.63s) **Cumulative Core Coverage: 98% (523 / 536 Statements)**

Module File	Stmts	Miss	Cover	Tested Scope & Functional Verification
backend/app.py	140	0	**100%**	FastAPI REST endpoints, input validation, graph payloads, proposals & static routes
backend/models.py	52	0	**100%**	Pydantic domain entities (User, SkillListing, Location, ScheduleWindow, TradeCycle)
backend/synthetic_data.py	54	0	**100%**	Dhaka synthetic population generator, skill taxonomy distribution, scalability benchmarks
backend/test_api.py	74	0	**100%**	Full integration suite verifying HTTP GET/POST/DELETE contracts & error paths
backend/test_engine.py	70	0	**100%**	Unit test harness for cycle bounds, Haversine ranking, DAG non-cycles & consensus states
backend/graph_engine.py	109	7	**94%**	Multigraph construction, bounded DFS cycle detection (k=3,4), composite scoring math
backend/workflow.py	37	6	**84%**	Atomic group proposal lifecycle, unanimous acceptance, single-rejection termination
TOTAL (Full Backend Core)	**536**	**13**	**98%**	**Comprehensive end-to-end algorithmic and integration verification**

Verification Environment: Python 3.14.6 | Pytest 9.1.1 | Coverage.py 7.16.0 | OS: Windows 11 AMD64 | All tests automated & reproducible via pytest --cov=backend | *RoundTable Exchange v2.4 (Single-Page Brief)*